

CS 11

Foundations of Program
Week 2 Lecture

Variable assignment & scope. Bo

Labor Day holiday

No Monday lab next week.

Handed out during lecture.

5 minutes remaining.

1. int, boolean, String types
2. Variable assignment
3. Variable scope
4. (break)
5. Handy terminal shortcuts
6. Common git commands: `clone` , `fetch/pull` , `status` ,
7. Lab: Push your first commit to Course-Information-Fall2026.

No class on Monday!

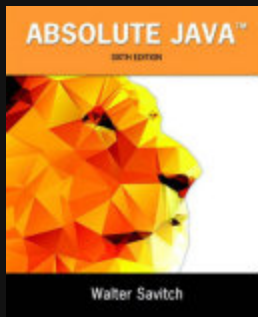
Quizzes will be handed back at the end of class **next Wednesday**

If you have any questions about assignments, learning resources, or anything else, please come to my office hours or contact me by email.

- Paria's office hours are now Thursdays, 11:30am - 1:30pm.
On Harvey Science 4th floor, in the CS tutoring lounge.
- Paria also holds tutoring hours from 1:30pm - 2:30pm Thursdays.
- **No office hours on Monday** for the holiday. Wednesday office hours will be moved to Tuesday.
- I will host remote office hours 1 - 3pm on Tuesday.
Sign up for a 15min slot during break or after class.

I realize the Head First Java 3rd edition text is poorly designed

Dr. Kristin Jones has a copy of Absolute Java 6th Edition by Walter Savitch which we can borrow during office hours.



Please come by my office hours to discuss anything related to

Literal values

A **literal** is a fixed value that you have written in your Java code.

- **integer** values: Numbers without a decimal place, including `0`.
- **boolean** values: Either `true` or `false`. Often used to control flow (e.g., `if` or `else`) or looping (`while` or `do/while` or `for`).
- **String** values: Text enclosed in straight double quotes (e.g., `"hello"`). To include a double-quote, escape it with a backslash (e.g., `"he said: \"hello\""`).

Expression values

In Java, an **expression** is a combination of variables, operators, and literals that produces a single value.

The **expression value** is the final result produced after Java carries out the necessary operations.

Every expression produces a value: Evaluating an expression can result in a value that is stored in a variable, passed as an argument, or used in a condition.

Expression Type

Example Expression

Arithmetic

`10 + 5 * 2`

Comparison

`15 > 20`

String Concatenation

`"Java " + 25`

Compound Expression

`(5 + 2) = 7`

Method Call

`"hello".toUpperCase()`

```
public class Comparison {  
    public static void main(String[] args) {  
        // These are variable assignments - we'll address them in a  
        int a = 5;  
        int b = 10;  
  
        System.out.println("Total: " + (a + b)); // Total: 15  
        System.out.println("Result: " + (a == b)); // Result: false  
    }  
}
```

Variable assignment in Java stores a value into a declared memory location. The value on the right-hand side is evaluated first and then stored in the variable.

```
int score = 100; // Declaration and initialization
score = 150;     // Reassigns 'score' to a new value
```

Mixing up variable assignment with equality tests is one of the most common mistakes in Java.

Assignment Operator (=)

Equality Test (==)

Action: Sets a value in a variable.

Question: Asks if two values are equal.

Returns the assigned value.

Returns a boolean value.

```
int x = 5;
```

```
if (x == 5)
```

Exercises

- **Types Must Match:** The variable type on the left must be correct (e.g., you cannot assign a `String` to an `int`).

```
public class AssignmentDemo {
    public static void main(String[] args) {
        boolean isReady = false;

        System.out.println("Initial value of isReady: " + isReady);
        System.out.println();

        if (isReady == true) {
            System.out.println("I can't be reached");
        } else {
            System.out.println("I'm working as expected");
        }
        System.out.println("Final value of isReady: " + isReady);
    }
}
```

- **Caution:** Assignment and equality operators look similar, but `(isReady = true) { ... }` will silently reassign `isReady` to